

COS214 Practical 2

Courtesy of

- Patrick Simuyemba: u25632354
- Azolile Mbang: u24820360

Task 1

1.1 Movement: MoveStrategy is trying to implement the Strategy design pattern, and it should rather be implemented as the State design pattern. Route: RouteState is trying to implement the State design pattern, and it should rather be implemented as the Strategy design pattern. Map: Map is trying to implement the Composite design pattern. Place features: The place features are trying to implement the Decorator design pattern. Biomes: WorldBuilder is trying to implement the Abstract Factory design pattern. The two labels that have been swapped is the Strategy and the State. Movement should be done with the State design pattern because an object should be able to change to a different move at runtime. The behaviour changes because the Traveller changes its state/mode. Route should be done with the Strategy design pattern because different route strategies use different algorithms to calculate the route. Choosing a route is interchangeable and changing the strategy changes how a route is calculated not the State of a trip.

1.2 Movement: Under the State pattern, the participants are the Client, Context, State and ConcreteStateN Route: Under the Strategy design pattern, the participants are the Client, Context, Strategy, ConcreteStrategyN Map: Under the Composite design pattern, the participants are the Client, Leaf, Composite, Component Place features: Under the Decorator design pattern, the participants are the Client, Component, ConcreteComponent, Decorator, ConcreteDecoratorA, ConcreteDecoratorB Biomes: Under the Abstract Factory design pattern, the participants are the Client, AbstractFactory, AbstractProduct, ConcreteFactory, ConcreteProduct

1.3 Wrong Pattern Choice: MoveStrategy is implementing the Strategy pattern when it should be implementing the State pattern. RouteState is implementing the State pattern when it should be implementing the Strategy pattern. The consequence in implementing this is the maintenance overhead that will come with adding new subclasses to the Strategy or State interfaces. The misuse creates unnecessary changes to existing classes when extending the system, increasing maintenance. Wrong UML Notation: The solid arrow used for the aggregation 'uses' relationship in the MoveStrategy relationship is incorrect because it is meant to be a solid line without an arrow. The <<Decorator>> label is not in a class diagram box, so it is not labelling anything. The maintenance consequence is that the intended pattern/class relationship is unclear. Broken ownership or missing virtual destructors: All the Location subclasses inherit from the Location class and no virtual destructor means that should the derived classes

have any dynamic memory allocated, it will not be freed when the object is destructed. The inheritance relationships with Location are flattened. That creates a maintenance overhead to manage each subclass with each added tag they have in their class name. Excessive Coupling: GameManager is dependent on all 5 subsystems which creates a massive overhead in terms of maintaining a GameManager object. The separation of responsibilities has been violated and changes to one subsystem could require a change to GameManager.

1.4 The GameManager class is the class that has been made to know everything. Coupling is how much classes depend on each other, and design patterns help to reduce high coupling and unnecessary dependencies by separating responsibilities and allowing components to interact through abstractions. GameManager should not centralise all the behaviour of the system. Each subsystem should manage what belongs to its responsibility. The Region class should own both other regions and the location objects the Composite participant in that subsystem. The WorldBuilder class should be the AbstractFactory participant responsible for creating the appropriate Concrete Factory for each biome. Movement and Route-finding should have their own Context classes with Traveller and Trip acting as those Context classes.

1.5 Arrows are overlapping the class diagrams and that interference makes it hard to follow where the relationships are and it is confusing to read the names of certain classes. The classes are arranged in a tightly packed manner on one side and then there is lots of white space and another section of the system. It is difficult to identify each subsystem because of how closely packed certain sections are. To improve the diagram, one could separate each subsystem to be evenly spaced around the central GameManager class. I would route the arrows around classes/components and not allow them to overlap with any other component.

Task 2 : Redesign

2.1 Completed

2.2 Design rationale

Pattern: Decorator

Participants:

- Component: PlaceFeature
- ConcreteComponent: Feature
- Decorator: FeatureDecorator
- ConcreteDecorator: Toll, Weather, Quest

Decision

- Aggregation on the

Pattern: Composite

[Participants]

- Component: Map
- Composite: Region
- Leaf: Location *Decision*
- this was what was decide yeah

Pattern: State *Participants*

- State: MovementState
- Context: MovementContext
- ConcreteState: Car, Air, Water, Foot *Decision*
- this was the decision

Pattern: Strategy *Participants*

- Strategy: RouteStrategy
- Context: RouteContext
- ConcreteStrategy: Fastest, Cheapest, Scenic, Shortest *Decision*
- this was my decicion

Pattern: AbstractFactory *Participants*

- AbstractFactory: WorldBuilder
- ConcreteFacotry: OceanBuilder, DesertBuilder,ForestBuilder, CityBuilder
- AbstractProduct: NPC, Biome, Obstacle
- ConcreteProduct:
 - DesertBiome, DesertObstacle, DesertNPC
 - OceanBiome, OceanObstacle, OceanNPC
 - ForestBiome, ForestObstacle, ForestNPC
 - CityBiome, CityObstacle, CityNPC

Decision

- This was the decision that was made

2.3 Additons

- Train:
- Water:
- Sn
- SnowBiome
- SnowNPC
- SnowObstacle